

AI Upskilling Platform

Sample Lesson Validation — 5 Lessons Across Types

CONTENTS



Reading

Object-Oriented Programming — Core Concepts

Generated 2026-05-29 · Powered by Claude claude-sonnet-4-6

Object-Oriented Programming — Core Concepts

Topic: Java Mastery

Overview

Before Object-Oriented Programming (OOP) became mainstream, developers wrote procedural code—long sequences of functions manipulating global data. This approach created brittle, hard-to-maintain systems. When you changed a data structure, you had to hunt through hundreds of functions that touched it. Variables lived everywhere, functions had unclear ownership over data, and reusing code meant copying it. OOP solves this by bundling data and the functions that operate on it into cohesive units called objects. Each object becomes a self-contained entity responsible for its own state and behavior. This encapsulation reduces bugs because changes to an object's internals don't ripple through unrelated code. Inheritance lets you build hierarchies where common behavior flows downward, eliminating duplication. Polymorphism allows objects to respond differently to the same message, enabling flexible, plugin-like architectures. Abstraction hides complexity behind simple interfaces. Together, these four pillars make systems that scale: a Java enterprise application managing millions of transactions, a game engine rendering thousands of game objects, or a microservice handling concurrent requests—all rely on OOP's power to organize complexity into manageable, reusable, maintainable chunks. Without OOP, modern software would be unmaintainable chaos.

✏️ Think of it like cricket: A cricket team is not just 11 players scattered randomly. During an India vs Australia test match at the MCG, Rohit Sharma opens the batting (data/role), backed by specific training routines and decision-making logic (methods/behavior). Each player—opener, middle-order batsman, all-rounder, fast bowler—has distinct responsibilities. Rohit Sharma's batting strategy differs from Jasprit Bumrah's bowling strategy, yet both are Players. Without OOP, you'd have one massive function called 'play_cricket' with 500 if-statements checking which player is doing what. With OOP, you create a Player class that encapsulates both identity (name, role, batting average) and behavior (bat(), bowl(), field()). Different player types inherit from Player but override methods—Rohit overrides bat() to be aggressive, while a tail-ender overrides it to be defensive. The team captain doesn't care how each player implements their role; she just calls player.performInMatch(), and polymorphism ensures the right behavior happens. This is exactly how OOP lets you design systems: define the contract (interface), let different objects implement it their own way, and compose them into working wholes. Understanding OOP through this lens reveals why encapsulation isn't just about 'hiding stuff'—it's about assigning clear ownership and responsibility, just like a cricket team's batting order assigns clear roles.

Core Concepts

Encapsulation

Encapsulation is the bundling of data (attributes) and methods (behavior) into a single unit—a class—and hiding the internal details behind a controlled interface. In Java, you achieve this using access modifiers: `private` restricts access to only the class itself, `protected` allows subclasses and same-package access, and `public` exposes the member to the world. When you mark a field `private`, you prevent external code from directly manipulating it. Instead, you provide public getter and setter methods that enforce rules. For example, if a `Player` object has a salary field, a setter could validate that the salary is non-negative before allowing the change. This validation would be impossible if external code could set salary directly. Encapsulation protects the object's invariants—the conditions that must always be true for the object to be in a valid state. It also allows you to change the internal implementation without breaking external code. If you later decide to compute salary from a formula instead of storing it directly, internal changes happen invisibly; code calling `getSalary()` continues working. This is why encapsulation is called 'data hiding'—not to prevent access, but to enforce correctness and maintain flexibility.

✏️ Think of it like cricket: A cricket team's scorecard is the public interface—runs, wickets, overs bowled. But the detailed internal mechanics of how a batsman scored those runs (defensive drive, aggressive pull, lucky edge)—that's private to the player and the match situation. Virat Kohli's training methods, his muscle memory, his strategic adjustments between deliveries—these are encapsulated within him. A commentator can report 'Kohli scored 45 off 60 balls' (the public interface) without knowing the private details of how he practiced or what his thoughts were. If Kohli changes his training routine next season, the public interface remains the same—he still plays innings and scores runs—but the internal implementation evolved. The cricket board doesn't expose raw data like 'Kohli's exact muscle activation patterns' to the public; it provides controlled information through a press briefing. Similarly, your Java class doesn't expose raw fields; it exposes controlled access through public methods. If someone tries to set Kohli's strike rate to -50 (invalid data), the encapsulation boundary should reject it—just as a team wouldn't allow a batsman's official stats to become negative. This reveals why encapsulation is powerful: it protects the object's integrity, hides irrelevant complexity, and lets you evolve internals freely.

Inheritance

Inheritance is a mechanism where a new class (called a subclass or derived class) acquires properties and methods from an existing class (called a superclass or base class). In Java, you declare inheritance using the `extends` keyword: `class Batsman extends Player`. The `Batsman` class inherits all non-private members from `Player`, including fields and methods. Inheritance models an 'is-a' relationship: a `Batsman` *is-a* `Player`. It eliminates code duplication by allowing shared behavior to live in the parent class. Both a `Batsman` and a `Bowler` might share a name, jersey number, and a method to report injury—these go in the `Player` superclass once, and both subclasses inherit them automatically. The subclass can also override inherited methods to provide specialized behavior. A `Bowler` might override the `play()` method to focus on bowling strategy, while a `Batsman` overrides it to

focus on batting strategy, yet both inherit the base attributes and can call super methods when needed. Inheritance also enables polymorphic code: if a method accepts a `Player` parameter, you can pass any subclass instance (`Batsman`, `Bowler`, `AllRounder`), and the method works with all of them. However, deep inheritance hierarchies become hard to navigate; Java uses single inheritance (a class extends only one parent) to keep things manageable, while interfaces provide a way to inherit multiple contracts.

🔪 Think of it like cricket: All cricketers—batsmen, bowlers, all-rounders—are `Players`. They all inherit common attributes: name, age, country, test record, fitness level. But a fast bowler like Jasprit Bumrah has specialized properties (bowling speed, economy rate, yorker accuracy) that a batsman doesn't have, and vice versa. Rohit Sharma (a batsman) inherits the `Player playMatch()` method, but he overrides it to implement aggressive batting strategy—attacking the new ball, playing powerful drives. Bumrah overrides the same `playMatch()` method to implement bowling strategy—setting field placements, varying pace and line. Both are `Players`, both have `playMatch()`, but each implements it differently. The cricket board can call `playMatch()` on any player without knowing whether they're a batsman or bowler; polymorphism handles it. If the board adds a new property to all players—say, 'international debut year'—all subclasses automatically gain it. Inheritance prevents repeating this property in `BatsmanStats`, `BowlerStats`, etc. However, if you create a 12-level inheritance chain (`Player > Cricketer > InternationalCricketer > TestCricketer > FastBowler > RightArmFastBowler > SeniorRightArmFastBowler...`), it becomes impossible to navigate and modify—this is why Java uses single inheritance. The insight: inheritance is powerful for sharing common traits and behavior, but deep chains become brittle; use composition and interfaces to keep hierarchies shallow and flexible.

Polymorphism

Polymorphism, derived from Greek words meaning 'many forms,' allows objects of different types to be treated through a common interface while executing type-specific behavior. In Java, polymorphism is achieved primarily through method overriding (runtime/dynamic polymorphism) and method overloading (compile-time/static polymorphism). Runtime polymorphism occurs when a subclass overrides a method defined in a superclass or interface. You declare a reference variable of the parent type but assign it an instance of a child type. For example, `Player player = new Batsman()` creates a `Batsman` but references it as a `Player`. When you call `player.performRole()`, Java looks at the actual runtime type (`Batsman`) and executes `Batsman`'s version of `performRole()`, not `Player`'s. This decouples code from specific types: a cricket match simulator can have a list of `Player` objects, call `performRole()` on each without knowing whether each element is a `Batsman`, `Bowler`, or `AllRounder`, and each object does the right thing. Method overloading is compile-time polymorphism: multiple methods share the same name but take different parameter types or counts. Java resolves which overloaded method to call at compile time based on argument types. Polymorphism enables extensibility—you can add new `Player` subclasses without modifying existing code that works with `Player` references. This is the Open/Closed Principle: classes should be open for extension but closed for modification. Without polymorphism, you'd need massive if-else chains: `if (player instanceof Batsman) { batsman code } else if (player instanceof Bowler) { bowler code }`, which breaks whenever you add a new player type.

✂ Think of it like cricket: The captain (cricket match simulator) sends a signal 'execute your assigned role in this over.' Every player—Rohit Sharma (batsman), Jasprit Bumrah (bowler), Hardik Pandya (all-rounder)—hears the same call but responds differently. Rohit takes guard at the crease and plays a delivery. Bumrah runs to the bowling crease and delivers the ball. Hardik might do either depending on the match phase. The captain doesn't think 'okay, Rohit is type Batsman, so call his batting routine; Bumrah is type Bowler, so call his bowling routine'—that would require the captain to know about every player type. Instead, all players inherit a common Player interface where `executeRole()` is defined. At runtime, Java looks at which actual player object it is and calls the right implementation. This is runtime polymorphism: the decision of what code to run happens at runtime based on the actual object type, not the reference type. Now imagine a match analyser that reviews 'how each player performed'—it loops through a list of Player objects (which are actually Batsmen, Bowlers, etc.), calls `performanceReport()` on each, and each player generates their own report (batting stats, bowling figures, fielding catches). The analyser never checks 'what type are you?'—polymorphism handles it automatically. This reveals why polymorphism is transformative: it lets you write flexible, extensible code that works with abstract types and automatically adapts to concrete implementations.

Abstraction

Abstraction is the process of hiding complex implementation details and exposing only essential features through a simplified interface. In Java, you achieve abstraction using abstract classes and interfaces. An abstract class is a class you cannot instantiate directly; it defines a contract that subclasses must implement. You mark a class as abstract with the `abstract` keyword and mark incomplete methods as `abstract` too. For example, `abstract class Player` might declare an abstract method `abstract void performRole()`, which every subclass must override. You never create a Player object directly; you create Batsman or Bowler objects, each implementing `performRole()` in their own way. An interface is a pure contract—a set of public method signatures with no implementation. A class implements an interface using the `implements` keyword. Interfaces enable a looser coupling than abstract classes because a class can implement multiple interfaces but extend only one class. Abstraction reduces cognitive load: users of a class need only understand its public methods and what they do, not how they do it. When you call `player.scoreRuns()`, you don't need to know the algorithm inside—whether it's calculating runs from delivery type, previous stats, or pure simulation logic. This separation of what from how allows developers to change implementations without breaking clients. Abstraction also enforces a contract: any class implementing the Scorer interface must provide `scoreRuns()`, ensuring consistency across different scoring implementations. At scale, abstraction is how teams collaborate: the API team defines interfaces, other teams implement them, and integration happens smoothly because everyone respects the contract.

✂ Think of it like cricket: The cricket rulebook (interface) defines abstract concepts: 'a batsman must face deliveries and accumulate runs.' It doesn't specify how Rohit Sharma personally scores runs—whether he plays drives, cuts, defensive pushes, or leaves the ball. The rulebook is abstract; the implementation is concrete and player-specific. Now imagine a match streaming app that shows 'runs scored this over'—viewers see the abstract result (15 runs scored) without seeing the complex internal details (what types of deliveries, what batting technique, what field placement exploited). The abstraction boundary is 'runs scored'; implementation details are hidden. A bowling coach studies Bumrah's bowling style in abstract terms—'vary pace, nail yorkers, set fields'—without needing to know the neurological details of how Bumrah's muscles coordinate. When the BCCI updates cricket regulations, they update the abstract rulebook (interface), and all player implementations adapt. Similarly, if you change your abstract Player interface to require a new method `recordStats()`, all implementing classes must add it—this ensures everyone follows the contract. Without abstraction, users of Player objects would need to know 300 things about the implementation; with abstraction, they know 5 things about the interface. This reveals why abstraction is essential at scale: it creates a barrier between what's internal (messy, changing, complex) and what's external (clean, stable, simple), making large systems comprehensible and maintainable.

java

```
// Abstraction: Define abstract Player class and Scorer interface
abstract class Player {
    // Encapsulation: private data with protected access for subclasses
    private String name;
    private int jerseyNumber;
    protected int matchesPlayed;

    public Player(String name, int jerseyNumber) {
        this.name = name;
        this.jerseyNumber = jerseyNumber;
        this.matchesPlayed = 0;
    }

    // Encapsulation: getter/setter with validation
    public String getName() {
        return name;
    }

    public void setName(String name) {
        if (name != null && !name.isEmpty()) {
            this.name = name;
        }
    }

    // Abstraction: abstract method that subclasses must implement
    abstract void performRole();

    // Concrete method inherited by all subclasses
    public void recordMatch() {
        matchesPlayed++;
    }
}

// Scoring interface: defines a contract for scorers
interface Scorer {
    void scoreRuns(int runs);
    int getTotalRuns();
}

// Inheritance + Polymorphism + Encapsulation + Abstraction
```

```

class Batsman extends Player implements Scorer {
    private int totalRuns;
    private int inningsCount;
    private double strikeRate;

    public Batsman(String name, int jerseyNumber) {
        super(name, jerseyNumber);
        this.totalRuns = 0;
        this.inningsCount = 0;
    }

    // Polymorphism: override abstract method from Player
    @Override
    void performRole() {
        System.out.println(getName() + " is batting at the crease.");
    }

    // Implement Scorer interface
    @Override
    public void scoreRuns(int runs) {
        if (runs ≥ 0) {
            totalRuns += runs;
        }
    }

    @Override
    public int getTotalRuns() {
        return totalRuns;
    }

    // Encapsulation: controlled access to strikeRate
    public void setStrikeRate(int ballsFaced) {
        if (ballsFaced > 0) {
            this.strikeRate = (double) totalRuns / ballsFaced * 100;
        }
    }

    public double getStrikeRate() {
        return strikeRate;
    }
}

```

```
// Another subclass: Bowler
class Bowler extends Player {
    private int wicketsTaken;
    private double economy;
    private int ballsBowled;

    public Bowler(String name, int jerseyNumber) {
        super(name, jerseyNumber);
        this.wicketsTaken = 0;
        this.ballsBowled = 0;
    }

    // Polymorphism: override abstract method from Player
    @Override
    void performRole() {
        System.out.println(getName() + " is bowling from the crease.");
    }

    // Encapsulation: setters with validation
    public void addWicket(int runs, int balls) {
        if (runs ≥ 0 && balls > 0) {
            wicketsTaken++;
            ballsBowled += balls;
            this.economy = (double) runs / (ballsBowled / 6.0);
        }
    }

    public int getWicketsTaken() {
        return wicketsTaken;
    }

    public double getEconomy() {
        return economy;
    }
}

// AllRounder: demonstrates inheritance from Player and interface implementation
class AllRounder extends Player implements Scorer {
    private int totalRuns;
    private int wicketsTaken;

    public AllRounder(String name, int jerseyNumber) {
```

```

        super(name, jerseyNumber);
        this.totalRuns = 0;
        this.wicketsTaken = 0;
    }

    @Override
    void performRole() {
        System.out.println(getName() + " is batting or bowling as needed.");
    }

    @Override
    public void scoreRuns(int runs) {
        if (runs >= 0) {
            totalRuns += runs;
        }
    }

    @Override
    public int getTotalRuns() {
        return totalRuns;
    }

    public void addWicket() {
        wicketsTaken++;
    }

    public int getWicketsTaken() {
        return wicketsTaken;
    }
}

// Main class: demonstrates polymorphism in action
public class CricketMatch {
    public static void main(String[] args) {
        // Create players: Inheritance + Encapsulation
        Batsman rohit = new Batsman("Rohit Sharma", 45);
        Bowler bumrah = new Bowler("Jasprit Bumrah", 93);
        AllRounder hardik = new AllRounder("Hardik Pandya", 31);

        // Polymorphism: treat all as Player despite different types
        Player[] players = {rohit, bumrah, hardik};
    }
}

```

```

// Call performRole() on each: polymorphism ensures right behavior
System.out.println("=== Match Start ===");
for (Player player : players) {
    player.performRole(); // Polymorphism: runtime decision
    player.recordMatch(); // Inherited method
}

// Encapsulation: interact through public interface
System.out.println("\n=== Batting Stats ===");
rohit.scoreRuns(45);
rohit.setStrikeRate(60); // 45 runs off 60 balls = 75% strike rate
System.out.println(rohit.getName() + ": " + rohit.getTotalRuns() +
    " runs, " + rohit.getStrikeRate() + "% SR");

// AllRounder implements Scorer interface
hardik.scoreRuns(28);
hardik.addWicket();
System.out.println(hardik.getName() + ": " + hardik.getTotalRuns() +
    " runs, " + hardik.getWicketsTaken() + " wicket(s)");

// Bowler encapsulation example
System.out.println("\n=== Bowling Stats ===");
bumrah.addWicket(32, 24); // 32 runs, 24 balls = 8.0 economy
System.out.println(bumrah.getName() + ": " + bumrah.getWicketsTaken() +
    " wickets, " + bumrah.getEconomy() + " economy");

// Polymorphic method: works with any Scorer
System.out.println("\n=== Scorers in Match ===");
Scorer[] scorers = {rohit, hardik};
for (Scorer scorer : scorers) {
    System.out.println("Total runs: " + scorer.getTotalRuns());
}
}
}

/* Output:
=== Match Start ===
Rohit Sharma is batting at the crease.
Jasprit Bumrah is bowling from the crease.
Hardik Pandya is batting or bowling as needed.

=== Batting Stats ===

```

```

Rohit Sharma: 45 runs, 75.0% SR
Hardik Pandya: 28 runs, 1 wicket(s)

== Bowling Stats ==
Jasprit Bumrah: 1 wickets, 8.0 economy

== Scorers in Match ==
Total runs: 45
Total runs: 28
*/

```

The code above demonstrates all four OOP pillars in concert. Encapsulation appears immediately: fields like `totalRuns` and `wicketsTaken` are private, accessed only through controlled public methods (`getTotalRuns()`, `setStrikeRate()`). The setters validate input—`scoreRuns(int)` only accepts non-negative values, preventing an invalid state where a batsman has negative runs. Inheritance is shown through `class Batsman extends Player`—Batsman inherits `name`, `jerseyNumber`, and `matchesPlayed` from `Player` without redeclaring them. The abstract class `Player` declares the abstract method `performRole()`, forcing every subclass to implement it; this is abstraction at work, defining a contract that all players must fulfill. Polymorphism shines in the main method: the array `Player[] players = {rohit, bumrah, hardik}` holds references of type `Player` pointing to `Batsman`, `Bowler`, and `AllRounder` objects. When the code calls `player.performRole()` in the loop, Java's runtime system executes the correct implementation: `Batsman.performRole()` for `rohit`, `Bowler.performRole()` for `bumrah`, `AllRounder.performRole()` for `hardik`. The code doesn't use if-else checks; polymorphism automatically dispatches to the right method. Similarly, the `Scorer` interface allows `rohit` and `hardik` to be treated as `Scorers` through the `scorers` array, even though they're different subclasses. When `scoreRuns(28)` is called on `hardik`, it adds runs to `AllRounder`'s `totalRuns` field (encapsulation), and `getTotalRuns()` retrieves the value through a public method, never directly exposing the field. This layering—abstraction hiding details, inheritance sharing code, polymorphism adapting behavior, encapsulation protecting state—creates a system that's extensible (add new `Player` types without touching existing code), maintainable (each class has clear responsibility), and type-safe (Java catches misuse at compile time).

 **Pro Tip:** Use composition over inheritance when designing complex systems. Instead of creating a deep `Player` → `Batsman` → `AggressiveBatsman` → `TestAggressiveBatsman` hierarchy, compose a `Player` with a `BattingStrategy` object. A `Player` has-a `BattingStrategy` rather than is-a specific type of `Batsman`. This keeps your class hierarchy flat, makes runtime strategy changes easier, and follows the Composition over Inheritance principle used in production frameworks like Spring and Hibernate.

 **Warning:** A common beginner mistake is overusing inheritance. Creating inheritance hierarchies deeper than 3-4 levels makes code hard to understand and modify. Every new level adds cognitive burden and makes the code fragile—changing the parent class might break all descendants. Instead, use interfaces to define contracts and let multiple unrelated classes implement them. If you catch yourself writing `abstract class AbstractBaseClassFactory`, it's usually a sign you should use composition, delegation, or interfaces instead.

Real-World Application

Object-Oriented Programming is the foundation of enterprise Java frameworks like Spring, Hibernate, and Jakarta EE. In a real production system serving millions of users—such as Netflix's content recommendation engine, Uber's ride-matching service, or a bank's transaction processing platform—OOP principles are essential. Spring Framework uses abstraction extensively: developers implement the Service interface without knowing that Spring's proxy system wraps your implementation to add transaction management, logging, and security checks. Polymorphism enables this magic: Spring doesn't care what concrete Service implementation you provide; it treats everything through the interface and applies cross-cutting concerns. Hibernate's ORM (Object-Relational Mapping) is built on inheritance and encapsulation: your `@Entity` classes inherit from a base persistent class, and Hibernate manages their SQL generation, caching, and lazy-loading transparently. Encapsulation protects domain objects: a User class hides its password field as private, exposing only a `validatePassword()` method, ensuring authentication logic isn't bypassed. In microservices architectures, each service exposes public interfaces (REST endpoints) that other services depend on—these interfaces are contracts, and services are polymorphic implementations of those contracts. When Netflix redesigned their architecture, services could evolve their internal implementation without affecting consumers as long as the interface remained stable. At Google, Facebook, and other scale-at-trillion-operations companies, OOP patterns like Abstract Factory (creating objects without specifying their concrete classes), Strategy Pattern (switching algorithms at runtime), and Observer Pattern (decoupling event sources from listeners) are used constantly. Without OOP's discipline, coordinating thousands of developers on codebases with millions of lines of code would be impossible. The design patterns and SOLID principles that govern enterprise code are all built on OOP foundations.

KEY POINTS

- Encapsulation bundles data and methods into objects while controlling access through private/public/protected keywords, protecting invariants and allowing internal changes without external impact.
- Inheritance allows subclasses to acquire and reuse code from parent classes through the 'is-a' relationship, reducing duplication and enabling overriding of methods for specialized behavior.
- Polymorphism lets different object types respond to the same method call with type-specific implementations, determined at runtime, eliminating if-else type checks and enabling extensible architectures.
- Abstraction hides complex implementation details behind simple public interfaces (abstract classes and interfaces), reducing cognitive load and allowing developers to reason about contracts rather than implementation.
- These four pillars work together: abstraction defines the contract, inheritance shares code, encapsulation protects state, and polymorphism enables flexible behavior—they're not isolated concepts but a unified design philosophy.
- In production systems like Spring, Hibernate, and microservices, OOP enables teams to build massive, maintainable codebases where services evolve independently without breaking consumers as long as contracts stay stable.
- The most common design mistake is creating deep inheritance hierarchies and overusing inheritance for code reuse; modern practice favors shallow hierarchies, composition, and interfaces for looser coupling and greater flexibility.

Q: You are designing a cricket match simulation. You have a `Player` abstract class with an abstract method `performRole()`. You create `Batsman` and `Bowler` as subclasses. In your main method, you create a `Player[]` array and assign `Batsman` and `Bowler` instances to it. When you loop through the array and call `performRole()` on each element, Java executes different code for `Batsman` vs `Bowler` without any if-else checks. Which OOP concept is this an example of?

A Encapsulation—because the `Player` class hides the `performRole()` method from external access

B Polymorphism—because the same method call (`performRole()`) executes different code depending on the actual runtime object type

C Inheritance—because `Batsman` and `Bowler` extend `Player` and inherit its structure

D Abstraction—because `performRole()` is declared as abstract in the `Player` class

Explanation: This is a textbook example of runtime polymorphism (also called dynamic dispatch or late binding). A `Player` reference variable points to a `Batsman` or `Bowler` object. When `performRole()` is called, Java looks at the actual object type at runtime (not the reference type) and executes the correct implementation. `Batsman.performRole()` and `Bowler.performRole()` are different, yet the code treats all objects uniformly. While inheritance, abstraction, and encapsulation are all present in this code, the core mechanism that makes the same method call behave differently based on object type is polymorphism. This is why polymorphism is transformative: it decouples code from concrete types and allows extensibility.

Q: You're building a user authentication system. A `User` class has a password field. If you make password public, any code anywhere can read or modify it directly, bypassing your validation logic. You want to ensure that password changes go through a secure setter that validates strength. Which OOP concept directly addresses this requirement?

A Polymorphism—because different `User` types can validate password differently

B Inheritance—because the `User` class can inherit validation behavior from a parent class

C Encapsulation—because it bundles the password field with a private modifier and exposes controlled access through a public setter method with validation

D Abstraction—because it hides the implementation details of password hashing

Explanation: This is a core example of encapsulation. By marking the password field as `private` and providing a public `setPassword(String newPassword)` method, you control how the password is modified. Inside the setter, you enforce rules: checking that the new password meets strength requirements, perhaps hashing it, checking against a breach database, and logging the change. External code cannot bypass these rules by directly assigning a value to the password field. Encapsulation protects the object's invariants—in this case, that the password field always contains a securely validated value. Without encapsulation, you'd have no way to prevent invalid states. While polymorphism might allow different user types to validate differently, and abstraction might hide implementation details, the core mechanism that prevents external code from directly accessing or modifying password is encapsulation through access control.